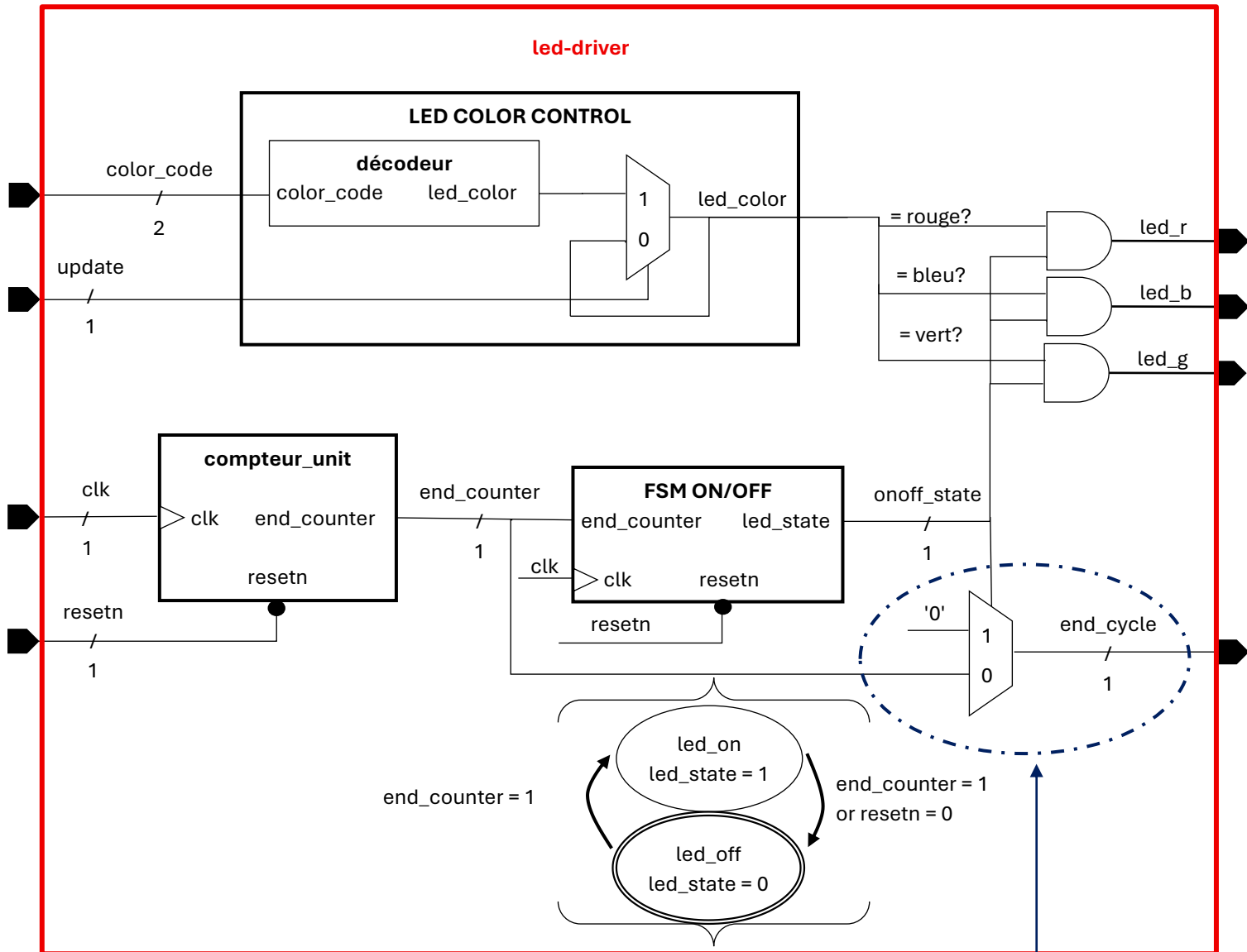


TP4 – Pilotage de LED et mémoire

1. Sur l'architecture RTL, modifiez le module LED_driver en ajoutant une sortie end_cycle. Cette sortie vaudra 1 à la fin d'un cycle allumé/éteint de la LED RGB.



La sortie end_cycle passera à 1 à la fin d'un cycle allumé/éteint.

end_cycle recopiera end_counter quand la LED sera en OFF (onoff_state = 0) grâce à un multiplexeur. Ceci permettra de générer un 1 sur end_cycle pendant une période d'horloge à la fin de chaque cycle.

- Implémentation VHDL

Le fichier est led_driver.vhd

Le fichier led_driver a été repris de TP4 partie1 et on a rajouté la ligne de description du multiplexeur qui génère end_cycle

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity led_driver is
    generic (
        cible : integer := 20
    );
    Port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          color_code : in STD_LOGIC_VECTOR (1 downto 0);
          update : in STD_LOGIC;
          end_cycle : out STD_LOGIC;
          led_r : out STD_LOGIC;
          led_g : out STD_LOGIC;
          led_b : out STD_LOGIC);
end led_driver;

architecture Behavioral of led_driver is
    type onoff_state is (led_on,led_off);
    signal current_onoff_state : onoff_state;
    signal next_onoff_state : onoff_state;
    type color_state is (off,rouge,bleu,vert);
    signal led_color : color_state;
    signal end_counter: std_logic:='0';

begin
    cpt: entity work.counter_unit(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            end_counter => end_counter
        );
    process(clk,resetn)
    begin
        if resetn = '0' then
            current_onoff_state <= led_off;
        elsif rising_edge(clk) then
            current_onoff_state <= next_onoff_state;
        end if ;
    end process;
end process;

```

```

-- FSM ON OFF
process(current_onoff_state, end_counter)
begin
    next_onoff_state <= current_onoff_state;
    case current_onoff_state is
        when led_off =>
            if end_counter = '1' then
                next_onoff_state <= led_on;
            end if;
        when led_on =>
            if end_counter = '1' then
                next_onoff_state <= led_off;
            end if;
        end case;
    end process;
-- LED COLOR CONTROL
process( update, color_code)
begin
    if update = '1' then
        case color_code is
            when "00" =>
                led_color <= off;
            when "01" =>
                led_color <= rouge;
            when "10" =>
                led_color <= vert;
            when "11" =>
                led_color <= bleu;
            when others =>
                led_color <= off;
        end case;
    end if ;
end process;
end_cycle <= end_counter when current_onoff_state = led_off else '0';
led_r <= '1' when led_color = rouge and current_onoff_state = led_on
else '0';
led_b <= '1' when led_color = bleu and current_onoff_state = led_on
else '0';
led_g <= '1' when led_color = vert and current_onoff_state = led_on
else '0';
end Behavioral;

```

Ajout du multiplexeur qui
gère le signal end_cycle



- Simulation et test bench

Le test_bench du TP4 partie 1 est repris afin de vérifier que rien n'a été cassé dans le led_driver (fichier tb_led_driver.vhd).
On rajoute à cette validation la vérification du signal end_cycle

a. Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, update, color_code

Liste des signaux de sortie à vérifier : led_r, led_b, led_g, **end_cycle**

Liste des signaux internes à vérifier : end_counter, led_color

Listes des états à vérifier: current_onoff_state, next_onoff_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

b. Déroulé de la validation et du test bench

1. Test du reset au démarrage

1.1. Les entrées seront définies comme suit :

- **resetsn = 0**
- update = 0
- color_code = 00

1.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g = 0
- **end_cycle = 0**

1.3. Les signaux : états internes à vérifier

- led_state = 0 resetté
- end_counter = 0 : counter_unit resetté
- led_color = off : led color control initialisé à off

1.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0,led_b=0, led_g=0 après $(1*(2*cible)+cible)*periode(clk)+1ns$
- on attend $cible*period + 1ns$ pour lancer la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **resetsn = 1**
- update = 0
- color_code = 00

2.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g = 0
- **end_cycle = end_counter quand current_onoff_state = led_off**

2.3. Les signaux : états internes à vérifier

- led_color est off , la LED est éteinte
- end_counter passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*periode(clk)$ secondes)

2.4. Sur le test bench

- on teste les sorties de led : led_r=0,led_b=0, led_g=0 après $cible*periode(clk)+1ns$
- on attend $2*2*cible*period + 1ns$ pour lancer la suite

3. Test de chaque code couleur avec son update

les étapes 1 à 3 seront répétées pour chaque code couleur

Je ne décris ici qu'une seule séquence pour le code '01': led rouge

Etape 1 : on applique le code couleur

3.1. Les entrées seront définies comme suit :

- resetsn = 1
- **color_code = '01'**
- update = '0'

3.2. Les sorties seront vérifiées

- led_r, led_b, led_g ne changent pas de fonctionnement
- **end_cycle = end_counter quand current_onoff_state = led_off**

3.3. Les signaux : états internes à vérifier

- led_color ne change pas d'état, il reste dans l'état où il était avant le changement de code couleur
- end_counter passe à 1 sur une période de clk quand le compteur_unit compte (cible -1)
- current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*periode(clk)$ secondes)

3.4. Sur le test bench

- on teste les sorties de led : led_r, led_b, led_g après 1ns: elle ne doivent pas avoir changé de comportement
- on attend $2*2*cible*period$ pour lancer la suite

4. Test de chaque code couleur avec son update

Etape 2 : on applique l'update

4.1. Les entrées seront définies comme suit :

- resetsn = 1

- `color_code = '01'`
 - **`update = '1'`**
- 4.2. Les sorties seront vérifiées
- **`led_r= 0/1`** à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
 - `led_b = '0', led_g = '0'`
 - **`end_cycle = end_counter quand current_onoff_state = led_off`**
- 4.3. Les signaux : états internes à vérifier
- **`led_color = rouge`**
 - `end_counter` passe à 1 sur une période de `clk` quand le compteur_unit compte (`cible -1`)
 - `current_onoff_state = led_off/led_on` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 4.4. Sur le test bench
- on teste les sorties de led : `led_r = 1 , led_b= 0 , led_g= 0`
 - on attend `2*2*cible*period` pour lancer la suite
- 5. Test de chaque code couleur avec son update**
- Etape 3 : on relâche l'update**
- 5.1. Les entrées seront définies comme suit :
- `resetsn =1`
 - `color_code = '01'`
 - **`update = '0'`**
- 5.2. Les sorties seront vérifiées
- **`led_r= 0/1`** à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
 - `led_b = '0', led_g = '0'`
 - **`end_cycle = end_counter quand current_onoff_state = led_off`**
- 5.3. Les signaux : états internes à vérifier
- **`led_color = rouge`**
 - `end_counter` passe à 1 sur une période de `clk` quand le compteur_unit compte (`cible -1`)
 - `current_onoff_state = led_off/led_on` à chaque fin de comptage du compteur_unit (la LED est allumée pendant `cible*period(clk)` secondes)
- 5.4. Sur le test bench
- on teste les sorties de led : `led_r = 1 , led_b= 0 , led_g= 0`
 - on attend `2*2*cible*period` pour lancer la suite
- 6. Test du reset sur un code couleur rouge**
- Nous avons auparavant rallumé la Led en rouge : `color_code = '01'` puis `update = 1`
- 6.1. Les entrées seront définies comme suit :
- **`resetsn = 0`**
 - `update = 1`
 - `color_code = 01`
- 6.2. Les sorties seront vérifiées
- `led_r = 0 , led_b = 0 , led_g= 0`
 - **`end_cycle = 0`**
- 6.3. Les signaux : états internes à vérifier
- `led_state = led_off` resetté
 - `end_counter = 0 : counter_unit` resetté
 - `led_color = rouge` : led color control n'est mis à jour que par l'update, pas de reset
- 6.4. Sur le test bench
- on teste les 3 sorties de led : `led_r =0,led_b= 0 , led_g= 0` après 1ns

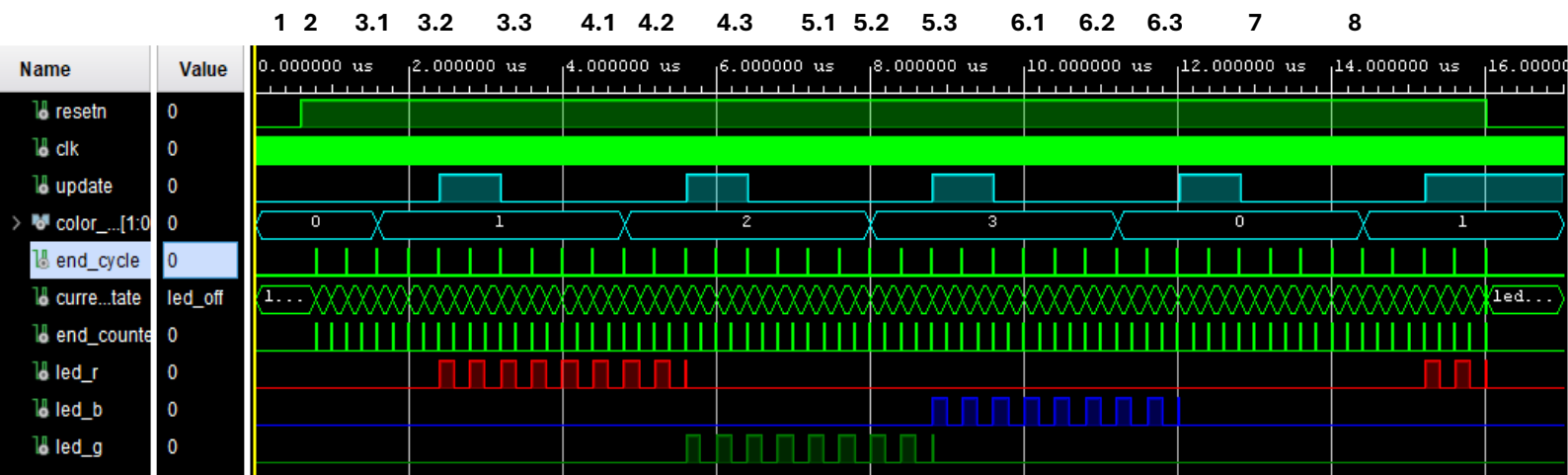
c. Test bench

Le fichier de test bench est `tb_led_driver.vhd`

d. Résultat des test automatiques du test_bench

```
# run 17000ns
Note: PASS : TEST reset : la led est eteinte OK
Time: 601 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST demarrage : la led est eteinte OK
Time: 802 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST code led rouge sans update : la led est eteinte OK
Time: 1603 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led rouge : la led est rouge OK
Time: 2404 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led rouge : la led reste rouge OK
Time: 3204 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST code led verte sans update : la led reste rouge OK
Time: 4805 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led verte : la led est verte OK
Time: 5606 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led verte : la led reste verte OK
Time: 6406 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST code led bleue sans update : la led reste verte OK
Time: 8007 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led bleue : la led est bleue OK
Time: 8808 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led bleue : la led reste bleue OK
Time: 9608 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST code led off sans update : la led reste bleue OK
Time: 11209 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led off : la led est eteinte OK
Time: 12010 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 0 pour code led off : la led reste eteinte OK
Time: 12810 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST code led rouge sans update : la led est eteinte OK
Time: 14411 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST update => 1 après code led rouge : la led est rouge OK
Time: 15212 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: PASS : TEST resetn => 0 : la led est eteinte OK
Time: 16013 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 16813 ns Iteration: 0 Process: /tb_led_driver/line__56 File: D:/Vivado/TP4_LED_P2/tb_led_driver.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 17000ns
| launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:09 . Memory (MB): peak = 1114.105 ; gain = 0.000
```

e. Validation fonctionnelle



1- Test du reset: la LED est éteinte (led_r = 0 , led_b= 0 , led_g = 0)

Le compteur_unit est resetté (end_counter = 0)

La FSM (current_onoff_state) est dans l'état led_off

La couleur de led est off (led_color = off)

end_cycle reste à 0

2- Démarrage après reset: le counter_unit démarre

la FSM (current_onoff_state) bascule de led_on à led_off à chaque fin de cycle de comptage (end_counter = 1)

La LED est off (led_color = off)

end_cycle = end_counter quand current_onoff_state = led_off

3.1- On applique le code couleur rouge

Rien ne change : la led est off (led_color = off)

end_cycle = end_counter quand current_onoff_state = led_off

3.2- On applique le update à 1

La LED clignote en rouge (led_r bascule de 0 à 1 , led_color = rouge)

end_cycle = end_counter quand current_onoff_state = led_off

3.3- On applique le update à 0

La LED continue à clignoter en rouge (led_r bascule de 0 à 1 , led_color = rouge)

end_cycle = end_counter quand current_onoff_state = led_off

4.1- On applique le code couleur vert

Rien ne change : La LED continue à clignoter en rouge

end_cycle = end_counter quand current_onoff_state = led_off

4.2- On applique le update à 1

La LED clignote en vert (led_g bascule de 0 à 1 , led_color = vert)

end_cycle = end_counter quand current_onoff_state = led_off

4.3- On applique le update à 0

La LED continue à clignoter en vert (led_g bascule de 0 à 1 , led_color = vert)

end_cycle = end_counter quand current_onoff_state = led_off

5.1- On applique le code couleur bleu

Rien ne change : La LED continue à clignoter en vert

end_cycle = end_counter quand current_onoff_state = led_off

5.2- On applique le update à 1

La LED clignote en bleu (led_b bascule de 0 à 1 , led_color = bleu)

end_cycle = end_counter quand current_onoff_state = led_off

5.3- On applique le update à 0

La LED continue à clignoter en bleu (led_b bascule de 0 à 1 , led_color = bleu)

end_cycle = end_counter quand current_onoff_state = led_off

6.1- On applique le code couleur off

Rien ne change : La LED continue à clignoter en bleu

end_cycle = end_counter quand current_onoff_state = led_off

6.2- On applique le update à 1

La LED est éteinte (led_color = off)

end_cycle = end_counter quand current_onoff_state = led_off

6.3 On applique le update à 0

La LED reste éteinte (led_color = off)

end_cycle = end_counter quand current_onoff_state = led_off

7 – On rallume la LED en rouge avant d'appliquer un reset

end_cycle = end_counter quand current_onoff_state = led_off

8 – On applique un reset alors que la LED clignote en rouge

La LED s'éteint

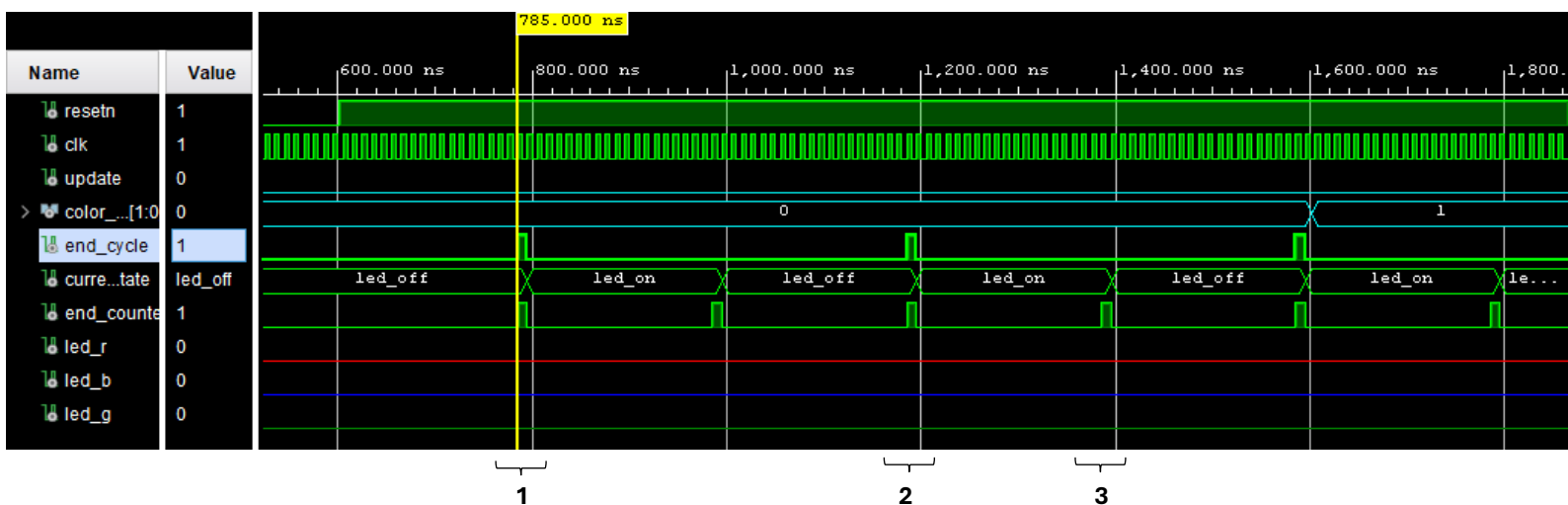
Le compteur_unit est resetté (end_counter = 0)

La FSM (current_onoff_state)est dans l'état led_off

La couleur de led reste à rouge (led_color = rouge)

end_cycle = 0

f. Zoom sur comportement de end_cycle



1 – Au démarrage après un reset , end_cycle passe à 1 pendant une période d'horloge sur le premier passage de la FSM ONOFF de OFF à ON

2 – end_cycle repasse à 1 pendant une période d'horloge sur le prochain passage de la FSM ONOFF de OFF à ON

3 – end_cycle reste à 0 sur les passages de la FSM ONOFF de ON à OFF

• Conclusion

La sortie end_cycle a été rajoutées au module led_driver. elle passe à 1 à la fin de chaque cycle allumé/éteint de la LED

2. Modifiez la logique en entrée du module pour ajouter une FIFO. Cette FIFO doit prendre en entrée le code couleur « vert » ou « bleu » suivant l'état du bouton_1 et est connectée en sortie à l'entrée color_code du module LED_driver. La donnée est écrite dans la FIFO lorsqu'il y a un front montant du bouton_0. La donnée de la FIFO est lue lorsque le signal end_cycle du module LED_driver vaut 1

- Génération de la FIFO à partir de IP catalog

IP Catalog

Cores | Interfaces

Q

≡

⚙

⚙

⚙

⚙

⚙

⚙

Search:

Name	AXI4	Status	License	VLNV
> Kernels				
> Math Functions				
> Memories & Storage Elements				
ECC		Production	Included	xilinx.com:ip:ecc:2.0
> FIFOs				
FIFO Generator	AXI4, AXI4-Stream	Production	Included	xilinx.com:ip:fifo_generator:13.2
URAM Read Back		Production	Included	xilinx.com:ip:uram_rd_back:1.0
> Memory Interface Generators				
> RAMs & ROMs				
> RAMs & ROMs & BRAM				

Details

Name:

FIFO Generator

Version:

13.2 (Rev. 5)

Interfaces:

AXI4, AXI4-Stream

Description:

The FIFO Generator is a parameterizable first-in/first-out memory queue generator. Use it to generate resource and performance optimized FIFOs with common or independent read/write clock domains, and optional fixed or programmable full and empty flags and handshaking signals. Choose from a selection of memory resource types for implementation. Optional Hamming code based error detection and correction as well as error injection capability for system test help to insure data integrity. FIFO width and depth are parameterizable, and for native interface FIFOs, asymmetric read and write port widths are also supported.

Status:

Production

License:

Included

Change Log:

[View Change Log](#)

Vendor:

Xilinx, Inc.

VLNV:

xilinx.com:ip:fifo_generator:13.2

Repository:

D:/Xilinx/Vivado/2020.2/data/ip

la FIFO vas être générée à partir de l'IP FIFO générateur

FIFO Generator (13.2)

[Documentation](#) [IP Location](#) [Switch to Defaults](#)☐ Show disabled ports

FIFO_WRITE

- full
- din[17:0]
- wr_en

FIFO_READ

- empty
- dout[17:0]
- rd_en

clk

srst

Component Name **Basic** **Native Ports** **Status Flags** **Data Counts** **Summary**

Interface Type

☒ Native ☐ AXI Memory Mapped ☐ AXI StreamFifo Implementation

la FIFO est générée avec
une clock commune pour
l'écriture et la lecture

FIFO Implementation Options

Supported Features

	Memory Type	(1)	(2)	(3)	(4)	(5)
Common Clock (CLK)	Block RAM	✓	✓		✓	✓
Common Clock (CLK)	Distributed RAM		✓			
Common Clock (CLK)	Shift Register					
Common Clock (CLK)	Built-in FIFO		✓	✓	✓	✓
Independent Clocks (RD_CLK, WR_CLK)	Block RAM	✓	✓		✓	✓
Independent Clocks (RD_CLK, WR_CLK)	Distributed RAM		✓			
Independent Clocks (RD_CLK, WR_CLK)	Built-in FIFO		✓	✓	✓	✓

(1) Non-symmetric aspect ratios (different read and write data widths)

(2) First-Word Fall-Through

(3) Uses Built-in FIFO primitives

(4) ECC support

(5) Dynamic Error Injection

FIFO Generator (13.2)

[Documentation](#) [IP Location](#) [Switch to Defaults](#)☐ Show disabled ports

FIFO_WRITE

- full
- din[1:0]
- wr_en

FIFO_READ

- empty
- dout[1:0]
- rd_en

clk

srst

Component Name **Basic** **Native Ports** **Status Flags** **Data Counts** **Summary**

Read Mode

☒ Standard FIFO ☐ First Word Fall Through

Data Port Parameters

Write Width 1,2,3,...1024

Write Depth Actual Write Depth: 32

Read Width

Read Depth Actual Read Depth: 32

le Bus d'entrée et sortie est
sur 2 bits (taille de
color_code)
on pourra stocker 32
données

ECC, Output Register and Power Gating Options

☐ ECC ☐ Single Bit Error Injection ☐ Double Bit Error Injection

☐ ECC Pipeline Reg ☐ Dynamic Power Gating

☐ Output Registers

Initialization

☒ Reset Pin

Reset Type

Full Flags Reset Value

☒ Dout Reset Value (Hex)

le reset est synchrone

la sortie sera à 0 lors du
reset

Read Latency: 1


☐ Show disabled ports
Component Name `led_driver_fifo`

Basic	Native Ports	Status Flags	Data Counts	Summary
-------	--------------	--------------	-------------	---------

Block RAM resource(s) (18K BRAMs): 1
 Block RAM resource(s) (36K BRAMs): 0

Clocking Scheme	Common Clock
Memory Type	Block RAM
Model Generated	Behavioral Model
Write Width	2
Write Depth	32
Read Width	2
Read Depth	32
Almost Full/Empty Flags	Not Selected/Not Selected
Programmable Full/Empty Flags	Not Selected/Not Selected
Data Count Outputs	Not Selected
Handshaking	Not Selected
Read Mode / Reset	Standard FIFO / Synchronous
Read Latency (From Rising Edge of Read Clock)	1

L'IP a été créée, les fichiers d'instanciation sont disponibles

Scope

Sources

?

—

□

⌕

IP (1)

led_driver_fifo (14)

Instantiation Template (2)

led_driver_fifo.vho

led_driver_fifo.vco

Synthesis (4)

Simulation (2)

Change Log (1)

led_driver_fifo.dcp

led_driver_fifo_sim_netlist.v

led_driver_fifo_sim_netlist.vhdl

led_driver_fifo_stub.v

led_driver_fifo_stub.vhdl

led_driver_fifo.vho

led_driver.vhd

tb_led_driver_behav.wcfg

top_led_driver.vhd

IP Catalog

d:/Vivado/led_fifo/gen/sources_1/ip/led_driver_fifo/led_driver_fifo.vho

52

-- The following code must appear in the VHDL architecture header.

53

----- Begin Cut here for COMPONENT Declaration ----- COMP_TAG

54

COMPONENT led_driver_fifo

55

PORT (

56

clk : IN STD_LOGIC;

57

srst : IN STD_LOGIC;

58

din : IN STD_LOGIC_VECTOR(1 DOWNTO 0);

59

wr_en : IN STD_LOGIC;

60

rd_en : IN STD_LOGIC;

61

dout : OUT STD_LOGIC_VECTOR(1 DOWNTO 0);

62

full : OUT STD_LOGIC;

63

empty : OUT STD_LOGIC

64

);

65

END COMPONENT;

66

-- COMP_TAG_END ----- End COMPONENT Declaration -----

67

68

69

-- The following code must appear in the VHDL architecture

70

-- body. Substitute your own instance name and net names.

71

72

----- Begin Cut here for INSTANTIATION Template ----- INST_TAG

73

your_instance_name : led_driver_fifo

74

PORT MAP (

75

clk => clk,

76

srst => srst,

77

din => din,

78

wr_en => wr_en,

79

rd_en => rd_en,

80

dout => dout,

81

full => full,

82

empty => empty

83

);

84

-- INST_TAG_END ----- End INSTANTIATION Template -----

--

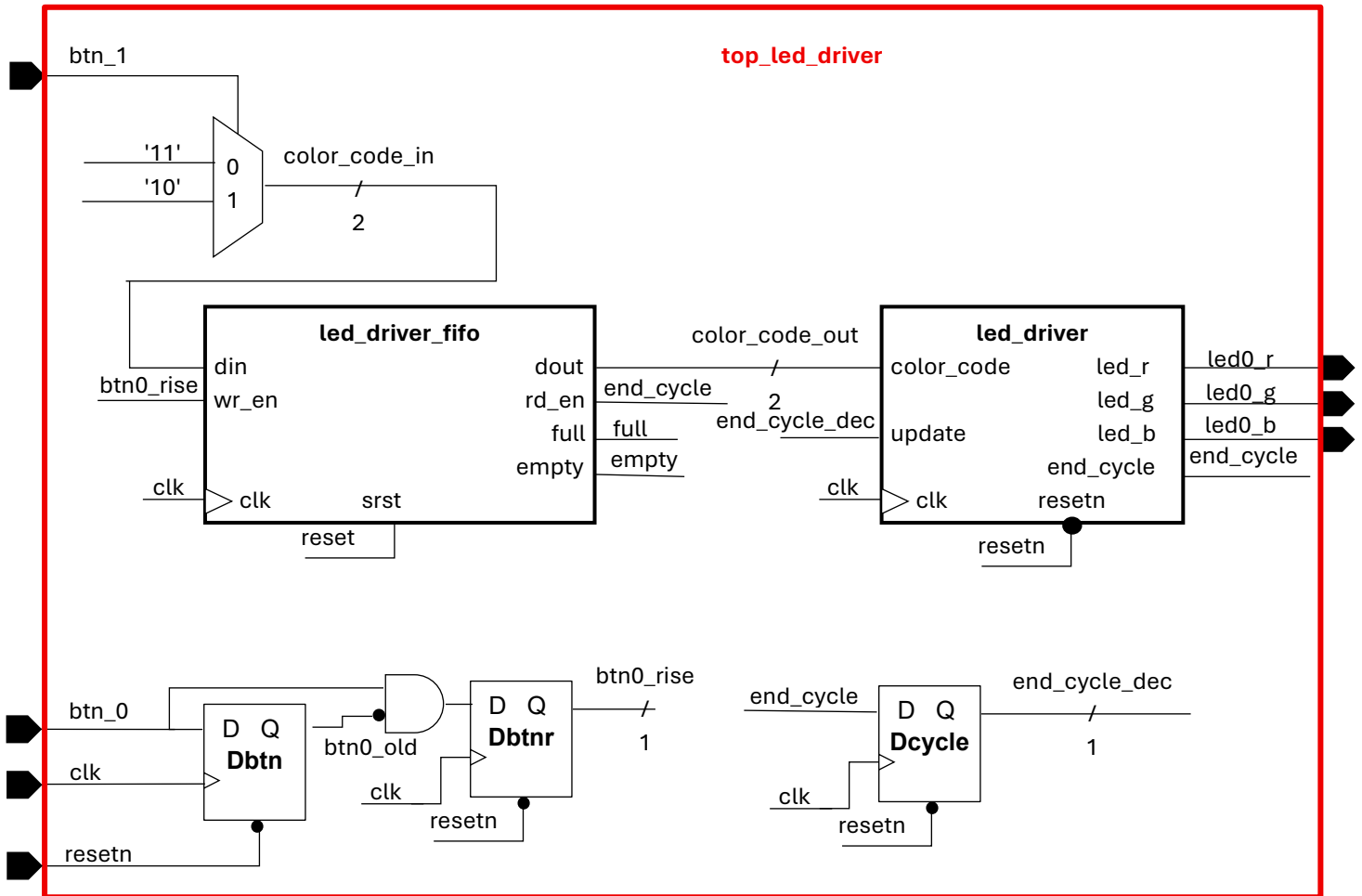
Hierarchy

IP Sources

Libraries

Compile Order

- Architecture du top_led_driver avec la FIFO



- 1- L'entrée de la FIFO est connectée au décodage de l'appuis sur le btn1 grâce au signal color_code_in
- 2- La donnée din est écrite sur les front montants du btn0: wr_en = btn0_rise
- 3- la sortie de la FIFO contrôle le code couleur du led_driver
- 4- la lecture de la FIFO est faite à chaque fin de cycle: rd_en= end_cycle
- 5- le led_driver est mis à jour un cycle d'horloge après la lecture de la fifo en raison de la latence de la fifo qui est de 1. On a donc décalé le signal end_cycle avec la bascule Dcycle et fournir le signal end_cycle_dec comme update du led_driver.

Note : les signaux full et empty de la FIFO ne sont pas utilisés dans cette architecture. Le signal empty sera juste contrôlé en simulation lors de l'écriture dans la FIFO pour vérifier que celle ci est bien écrite

- Implémentation du VHDL

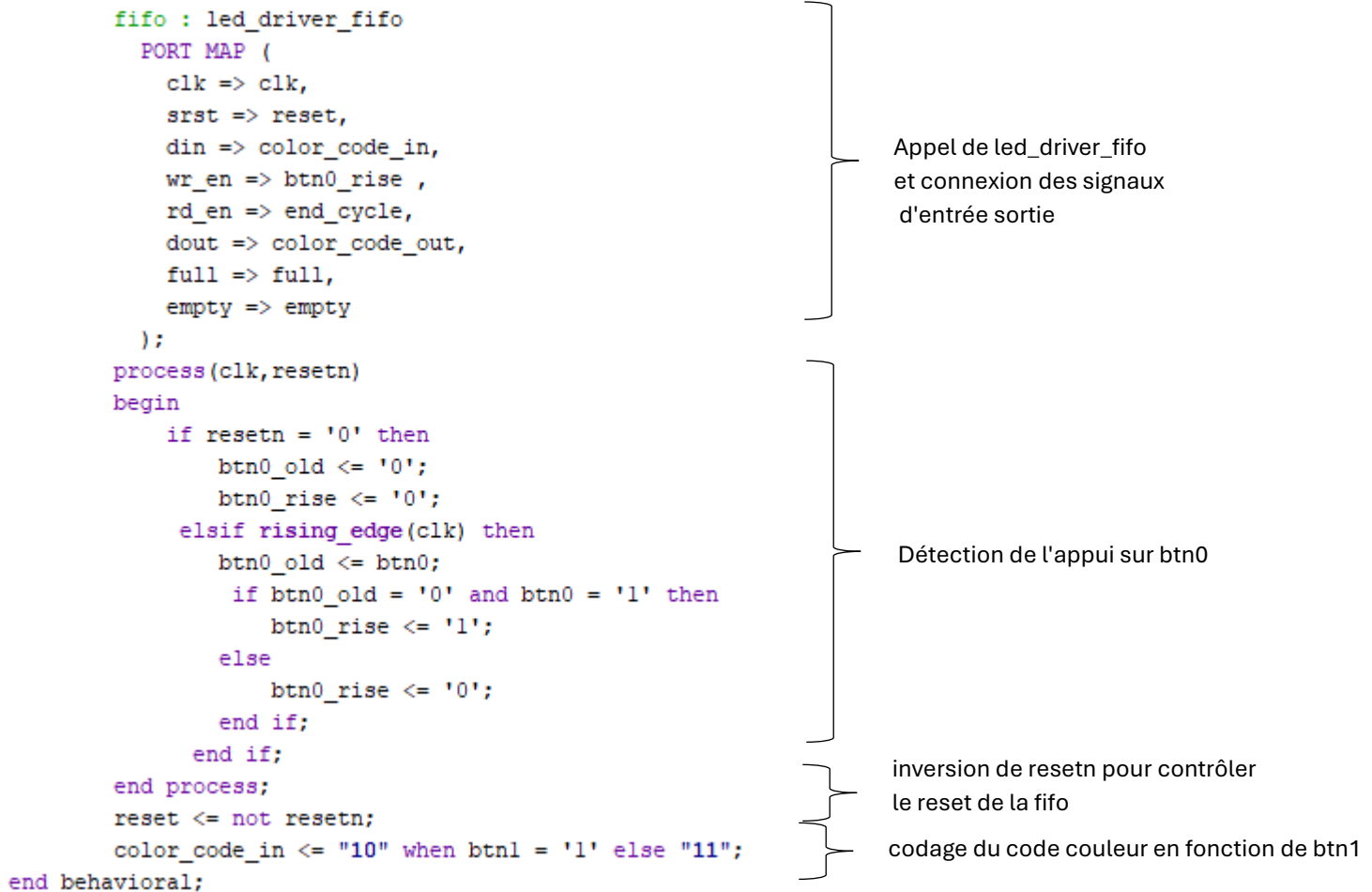
le fichier est top_led_driver.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_led_driver is
    generic (
        cible : integer := 2000000000
    );
    port (
        clk      : in std_logic;
        resetn   : in std_logic;
        btn0     : in std_logic;
        btn1     : in std_logic;
        led0_r   : out std_logic;
        led0_b   : out std_logic;
        led0_g   : out std_logic
    );
end top_led_driver;
```

```
architecture behavioral of top_led_driver is
    signal color_code_in : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal color_code_out : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal end_cycle : std_logic := '0';
    signal btn0_old : std_logic := '0';
    signal btn0_rise : std_logic := '0';
    signal full : std_logic := '0';
    signal empty : std_logic := '0';
    signal reset : std_logic := '0';
    component led_driver_fifo
    port (
        clk      : in std_logic;
        srst     : in std_logic;
        din      : in std_logic_vector(1 downto 0);
        wr_en    : in std_logic;
        rd_en    : in std_logic;
        dout     : out std_logic_vector(1 downto 0);
        full     : out std_logic;
        empty    : out std_logic
    );
    end component;
begin
    led_driver: entity work.led_driver(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            color_code => color_code_out,
            update => end_cycle,
            end_cycle => end_cycle,
            led_r  => led0_r,
            led_g  => led0_g,
            led_b  => led0_b
        );
```

déclaration de la fifo
led_driver_fifo

Appel de led_driver
et connexion des signaux
d'entrée sortie



- Validation et test bench

- a. Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn, btn0, btn1

Liste des signaux de sortie à vérifier : led_r, led_b, led_g

Liste des signaux internes à vérifier : color_code_in, color_code_out, end_cycle, btn0_rise, led_driver: led_color

Listes des états à vérifier: led_driver: current_onoff_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- b. Déroulé de la validation et du test bench

Le but de cette validation est d'enregistrer une séquence de couleur dans la fifo et vérifier que la LED clignote bien avec cette même séquence en sortie

- 1. Test du reset au démarrage

- 1.1. Les entrées seront définies comme suit ::

- **resetn = 0**
 - btn0 = 0
 - btn1 = 0

- 1.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g= 0

- 1.3. Les signaux : états internes à vérifier

- color_code_in = '11' (valeur pour btn1 =0)
 - color_code_out = 0 (fifo resetté à zéro)
 - end_cycle = 0
 - btn0_rise = 0
 - led_driver: led_color = off lors du reset
 - led_driver: current_onoff_state = led_off resetté

1.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0, led_b= 0, led_g= 0 après $(1*(2*cible)+cible)*periode(clk)+1ns$
- on attend $cible*period + 1ns$ pour lancer la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **resetn = 1**
- btn0 = 0
- btn1 = 0

2.2. Les sorties seront vérifiées

- led_r = 0 , led_b = 0 , led_g= 0

2.3. Les signaux : états internes à vérifier

- color_code_in = '11' (valeur pour btn1 =0)
- color_code_out = 0
- **end_cycle passe à 1 sur une periode de clk toutes les $2*cible*periode(clk)$**
- btn0_rise = 0
- led_driver: led_color = off
- **led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)**

2.4. Sur le test bench

- on teste les 3 sorties de led : led_r=0, led_b= 0, led_g= 0 après $cible*periode(clk)+1ns$
- on attend $cible*period + 1ns$ pour lancer la suite

3. Enregistrement et lecture d'une séquence

3.1. Les entrées seront définies comme suit :

- resetn = 0
- **btn0 = 0/1/0 entre chaque changement de valeur de btn1 pour charger la fifo (l'appui sur le btn0 durera une periode d'horloge)**
- **btn1 suivra cette séquence : 1/0/1/0/0/1/0/1/1/0/1/0/1 ce qui correspond à V/B/V/B/B/V/B/V/V/B/V/B/V**

3.2. Les sorties seront vérifiées

- led_r = 0 ,
- **led_b = 0/1 quand la sortie de la fifo sera color_code_out = '11' sinon 0**
- **led_g= 0/1 quand la sortie de la fifo sera color_code_out = '01' sinon 0**

3.3. Les signaux : états internes à vérifier

- **color_code_in suivra la séquence de programmation**
- **color_code_out sa valeur suivra la séquence avec une mise à jour toutes les $2*cible*periode(clk)$ sur end_cycle**
- end_cycle passe à 1 sur une periode de clk toutes les $2*cible*periode(clk)$
- **btn0_rise détectera l'appui sur el bouton est passera à 1 sur une periode d'horloge après l'appui sur btn0**
- **led_driver: led_color suivra la valeur du color_code_out sur end_cycle_dec**
- led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant $cible*period(clk)$ secondes)

3.4. Sur le test bench

- on déclare une sequence (vert/bleu) dans un tableau
- process 1 : une boucle parcourt le tableau de séquence et actionne btn1 suivant la valeur dans le tableau de séquence puis on simule un appui sur btn 0 pour écrire dans la fifo.
- process 2 : on attend le premier changement de couleur selon ce qui est prévu dans le tableau de séquence. Puis on vérifie toutes les $2*cycle*period(clk)$ que les sorties de LED sont conforme a la séquence définie.
- note: il faut faire deux process car le déroulement des écritures et des lectures dans la fifo ne sont pas séquentiels mais dépendent de deux temporisations différentes.
- on attend la fin de la lecture de la séquence pour lancer la suite

4. Test du reset en fin de séquence

4.1. Les entrées seront définies comme suit :

- **resetn = 0**
- btn0 = 0

- btn1 = 0
- 4.2. Les sorties seront vérifiées
 - led_r = 0 , led_b = 0 , led_g= 0
- 4.3. Les signaux : états internes à vérifier
 - color_code_in = '10' (valeur pour btn1 =1)
 - color_code_out = 0 (fifo resetté à zéro)
 - end_cycle = 0
 - btn0_rise = 0
 - led_driver: led_color = vert reste sur la dernière couleur de la séquence
 - led_driver: current_onoff_state = led_off resetté
- 4.4. Sur le test bench
 - on teste les 3 sorties de led : led_r=0,led_b= 0, led_g= 0 après periode(clk)+1ns
 - on attend 2*cible*period +1ns pour lancer la suite
- 5. Test du démarrage après reset**
 - 5.1. Les entrées seront définies comme suit : :
 - **resetn = 1**
 - btn0 = 0
 - btn1 = 1
 - 5.2. Les sorties seront vérifiées
 - led_r = 0 , led_b = 0 , led_g= 0
 - 5.3. Les signaux : états internes à vérifier
 - color_code_in = '10' (valeur pour btn1 =1)
 - color_code_out = 0
 - **end_cycle passe à 1 sur une periode de clk toutes les 2*cible*periode(clk)**
 - btn0_rise = 0
 - led_driver: led_color = off
 - **led_driver: current_onoff_state = led_off/led_on à chaque fin de comptage du compteur_unit (la LED est allumée pendant cible*period(clk) secondes)**
 - 5.4. Sur le test bench
 - on teste les 3 sorties de led : led_r=0,led_b= 0, led_g= 0 après cible*periode(clk)+1ns
 - le test est fini

c. Résultat du test bench

le fichier de test bench est tb_top_led_driver.vhd

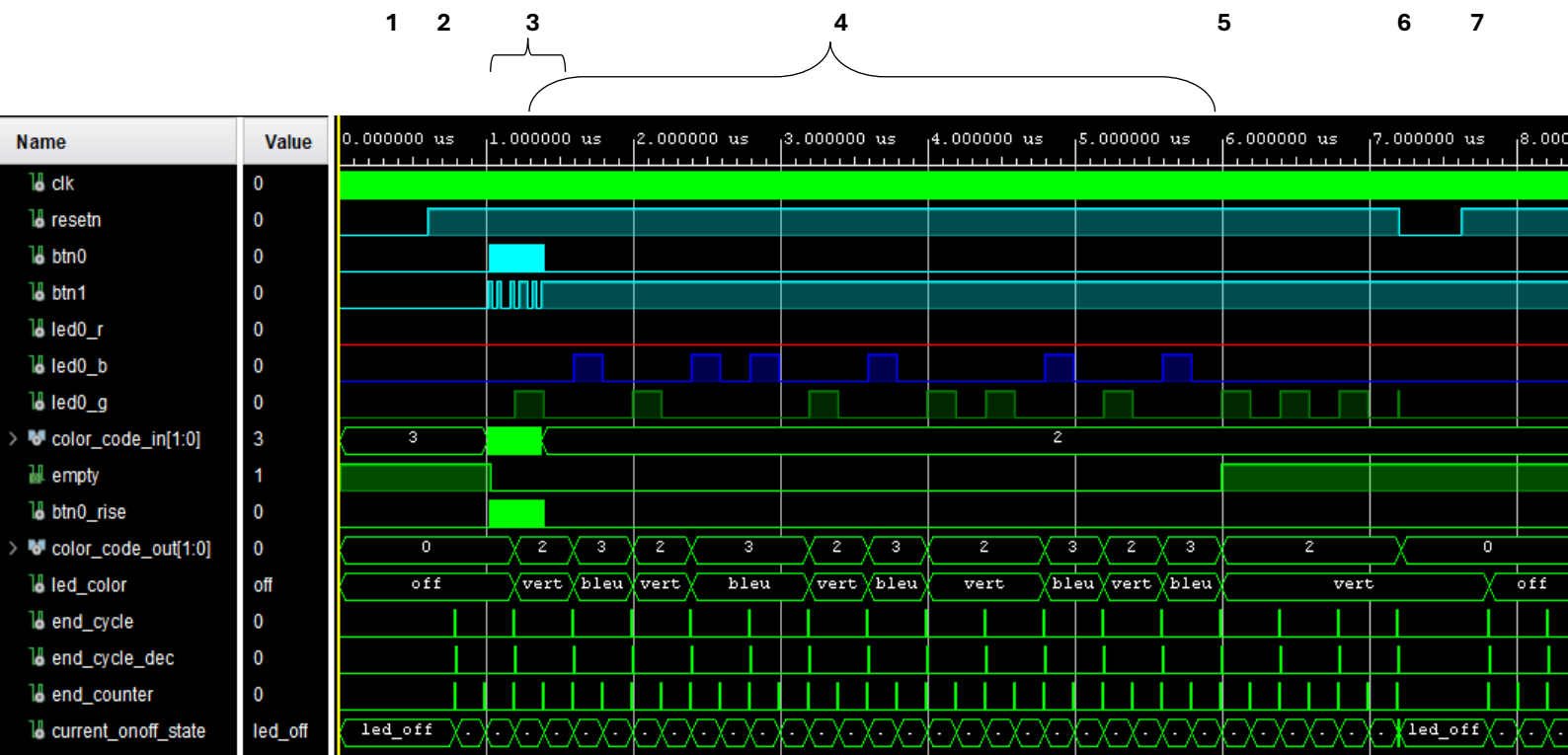
le check automatique vérifie que la séquence écrite est bien affichée sur la LED

```

# run 9000ns
Note: PASS : TEST reset : la led est eteinte OK
Time: 601 ns Iteration: 0 Process: /tb_top_led_driver/line__79 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS : TEST demarrage : la led est eteinte OK
Time: 802 ns Iteration: 0 Process: /tb_top_led_driver/line__79 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 0
Time: 1195 ns Iteration: 10 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED bleue allumée pour le cycle 1
Time: 1596 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 2
Time: 1997 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED bleue allumée pour le cycle 3
Time: 2398 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED bleue allumée pour le cycle 4
Time: 2799 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 5
Time: 3200 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED bleue allumée pour le cycle 6
Time: 3601 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 7
Time: 4002 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 8
Time: 4403 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED bleue allumée pour le cycle 9
Time: 4804 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 10
Time: 5205 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED bleue allumée pour le cycle 11
Time: 5606 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS LED verte allumée pour le cycle 12
Time: 6007 ns Iteration: 0 Process: /tb_top_led_driver/line__137 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS : TEST reset : la led est eteinte OK
Time: 7220 ns Iteration: 0 Process: /tb_top_led_driver/line__79 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: PASS : TEST demarrage : la led est eteinte OK
Time: 7822 ns Iteration: 0 Process: /tb_top_led_driver/line__79 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 8223 ns Iteration: 0 Process: /tb_top_led_driver/line__79 File: D:/Vivado/TP4_LED_P2/tb_top_led_driver.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_led_driver_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 9000ns
) launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:10 . Memory (MB): peak = 1343.164 ; gain = 0.000

```

- Validation fonctionnelle



1-Test du reset: la LED est éteinte (led_r = 0 , led_b = 0 , led_g = 0)

La FSM (current_onoff_state)est dans l'état led_off

La couleur de led est off (led_color = off)

L'entrée de la fifo color_code_in est confirme ai niveau sur btn 1 (btn1 = 0 => color_code_in = "11")

La sortie de la fifo color_code_out est à zéro

Le led_driver est resetté : end_cycle = 0

Les registres sont resetté btn0_rise = 0, end_cycle_dec = 0

2- Démarrage après reset: le counter_unit démarre

la FSM(current_onoff_state) bascule de led_on à led_off à chaque fin de cycle de comptage (end_counter = 1)

La LED est off (led_color = off)

3-Ecriture dans la fifo

La séquence est déroulé sur btn1

btn0 passe à 1 à chaque séquence

Le code couleur color_code_in en entrée de la fifo est mis à jour.

le signal empty de la fifo passe à 0 en début d'écriture pour signaler que la fifo contient bien des données

4-Lecture de la fifo

La lecture est activé pas le signal end_cycle tous les 2 cycles de end_counter

La séquence est déroulée sur la sortie de la fifo color_code_out

La LED s'éclaire selon la couleur de la séquence

5-Fin de lecture

En fin de séquence la fifo est vide (empty = 1)

La sortie de la fifo reste sur la dernière donnée de la séquence (btn1 = 1, LED verte)

La LED continue de clignoter en vert

6-Reset

La LED s'éteint

La fifo est resettée (color_code_out = 0)

Le compteur de cycle est resetté (end_counter = 0)

Le led_driver est resetté (end_cycle = 0)

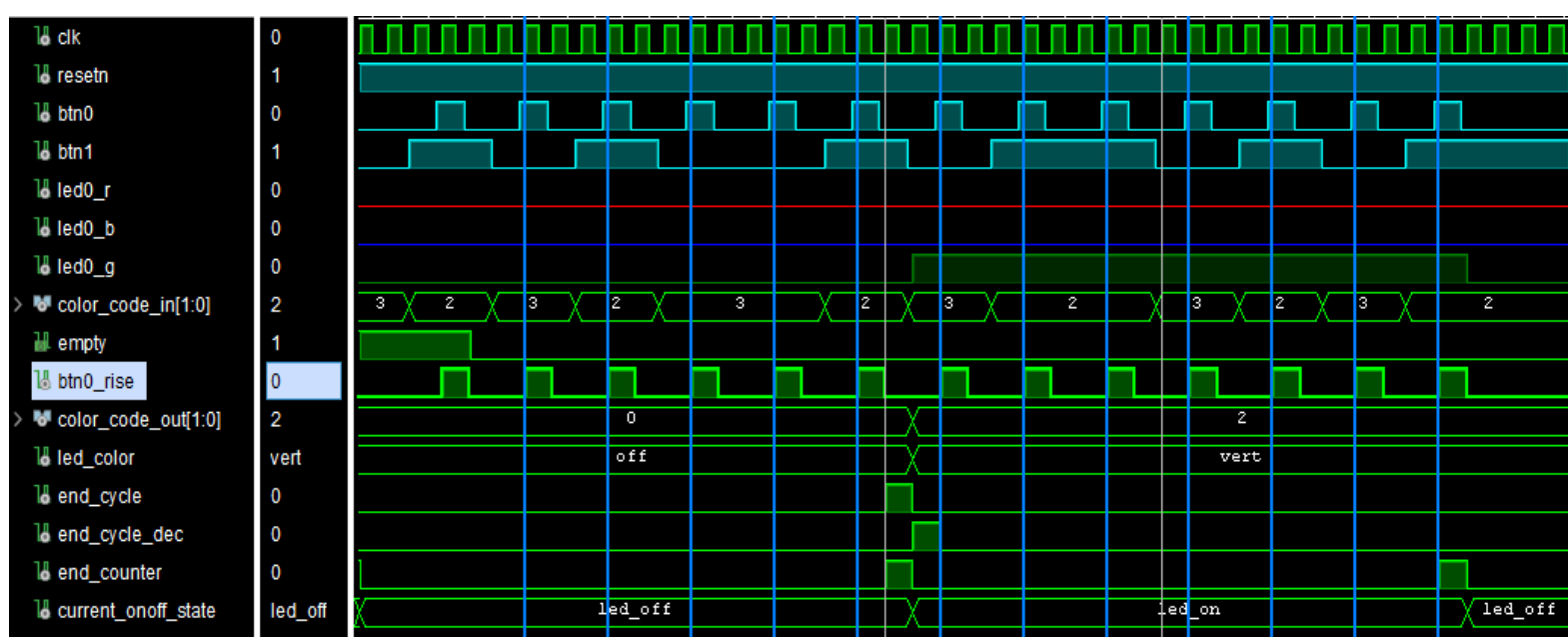
Le signal led_color n'est pas mis à zéro car il n'est pas resetté par le signal resetn

7- Redémarrage après reset

Le compteur de cycle redémarre end_counter = 1 toutes les fin de cycle de comptage et end_cycle = 1 tous les 2 cycles de comptage

La LED est éteinte . Le signal led_color qui contrôle la couleur de la LED passe à OFF car au premier update le LED_driver (end_cycle_dec = 1) la sortie de la fifo est à 0 color_code_out = 00 donc led_color = OFF

- Validation écriture de la fifo



L'appui sur les boutons btn 0 et btn1 se fait de manière asynchrone

Lors d'un front montant sur btn0 , btn0_rise passe à 1 sur une période d'horloge

L'écriture dans la fifo est faite une période d'horloge après le write enable en = btn0_rise, à ce moment là empty = 0 indiquant que la fifo n'est pas vide.

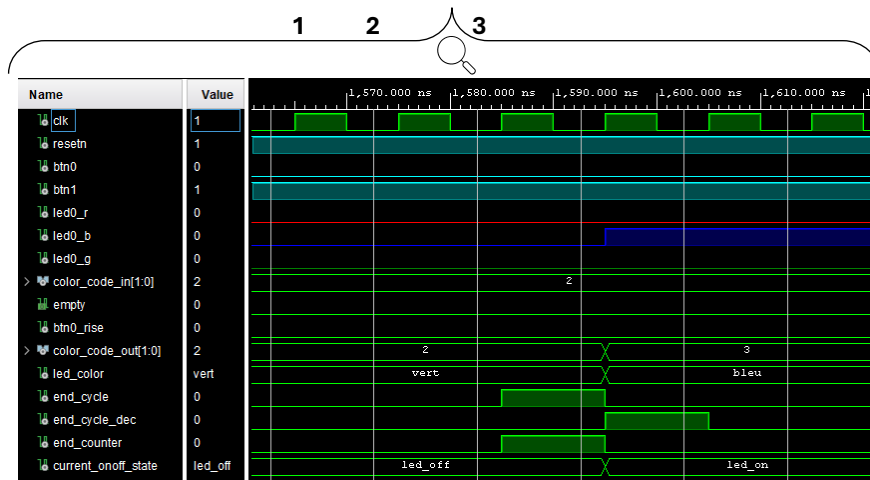
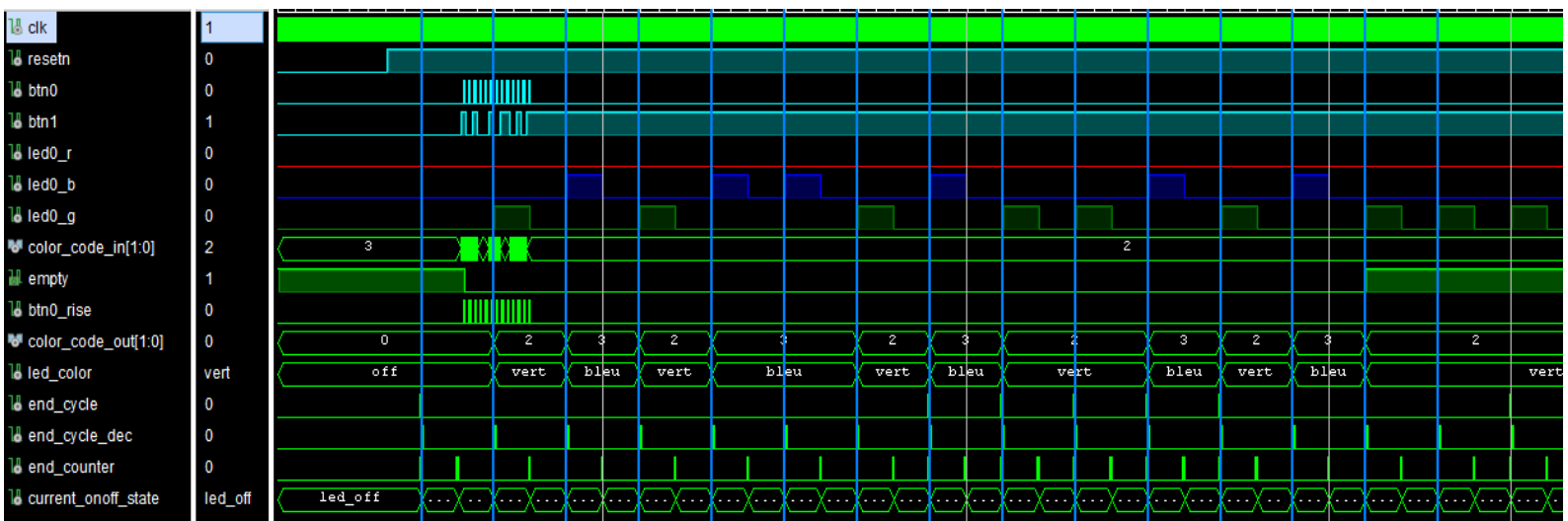
la fifo se remplit avec la séquence définie à chaque appui sur btn0 (btn0_rise = 1)

on retrouve bien sur le signal color_code_in la séquence prévue dans le test bench

2/3/2/3/3/2/3/2/3/2/3/2

V/B/V/B/V/B/V/V/V/B/V/B/V

- Validation lecture de la fifo et drive de la LED



1- Au démarrage : end_cycle passe à 1 sur la première transition off_on mais la fifo est vide à ce moment-là. Le sortie de la fifo est à 0 , le LED est éteinte

2- La première lecture du premier élément stocké dans la fifo intervient au prochain cycle (au prochain end_cycle = 1)
La sortie de la fifo donne le premier élément l de la séquence stocké color_code_out = 01 pour la couleur verte.
La LED passe ne vert quand end_cycle_dec passe à 1

3- Zoom sur une lecture :

La lecture est activée quand end_cycle passe à 1.

end_cycle passe à 1 lors du changement de cycle : current_onoff_state de led_off vers led_on

La sortie de la fifo prend alors la valeur stockée suivante. color_code_out change de valeur sur la période d'horloge suivante en raison de la latence à 1 de la fifo.

La mise à jour du led_driver est activée par end_cycle_dec (signal end_cycle décalé d'un coup d'horloge), la couleur de la LED est alors mise à jour.

4- La lecture du dernier élément de la séquence est indiqué par le signal empty = 1

La sortie de la fifo indique le dernier élément de la séquence color_code_out = 01.

Ensuite pour les prochaines commandes de lecture (end_cycle = 1) la sortie de la fifo indiquera toujours ce dernier élément. La LED continue de clignoter en vert

Check de la séquence lue : on retrouve bien sur le signal color_code_out la séquence prévue dans le test bench

2/3/2/3/3/2/3/2/3/2/3/2

V/B/V/B/V/B/V/B/V/B/V/B/V

5. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.

- Définition du fichier de contraintes

```
# PL System Clock
set_property -dict {PACKAGE_PIN H16 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -period 8.000 -name sys_clk_pin -waveform {0.000 4.000} -add [get_ports clk]

# RGB LEDs
set_property -dict {PACKAGE_PIN L15 IOSTANDARD LVCMOS33} [get_ports led0_b]
set_property -dict {PACKAGE_PIN G17 IOSTANDARD LVCMOS33} [get_ports led0_g]
set_property -dict {PACKAGE_PIN N15 IOSTANDARD LVCMOS33} [get_ports led0_r]
#set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
#set_property -dict { PACKAGE_PIN L14 IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
#set_property -dict { PACKAGE_PIN M15 IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r

# Buttons
set_property -dict {PACKAGE_PIN D20 IOSTANDARD LVCMOS33} [get_ports btn0]
set_property -dict {PACKAGE_PIN D19 IOSTANDARD LVCMOS33} [get_ports btn1]

## User Digital I/O Header J1
set_property -dict {PACKAGE_PIN L19 IOSTANDARD LVCMOS33} [get_ports resetn]
#set_property -dict { PACKAGE_PIN M19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[2] }]; #IO_L7P_T1_AD2P_35 Sch=user_dio[2]
```

La fréquence de l'horloge sur la carte est de 125Mhz soit une période de 8ns.

le signal de resetn est routé sur un user_io[1] pour pouvoir être connecté à une alimentation sur la carte

- Résultat de la synthèse

Detailed RTL Component Info :	
-----Adders :	1 additionneur du counter_unit sur 28 bits
2 Input 28 Bit Adders := 1	
-----Registers :	1 registre 28 bits pour la cible du counter_unit
28 Bit Registers := 1	1 registre 1 bit pour la machine d'état onoff,
1 Bit Registers := 4	2 registres 1 bit pour la gestion de front montant de btn0
-----Muxes :	1 registre pour le décalage du end_cycle
2 Input 28 Bit Muxes := 1	1 mux 28 bits pour le counter_unit
4 Input 2 Bit Muxes := 1	1 mux 2 bits 4 entrées pour le décodeur de code couleur
2 Input 2 Bit Muxes := 1	1 mux 2 bit 2 entrées pour le codage du code couleur
2 Input 1 Bit Muxes := 1	1 mux 1 bit 2 entrées pour la machine d'état onoff

Report BlackBoxes:

	BlackBox name	Instances
1	led_driver_fifo	1

Report Cell Usage:

	Cell	Count
1	led_driver_fifo	1
2	BUFG	1
3	CARRY4	7
4	LUT1	2
5	LUT2	2
6	LUT3	3
7	LUT4	27
8	LUT5	2
9	LUT6	1
10	FDCE	31
11	FDPE	1
12	LD	2
13	IBUF	4
14	OBUF	3

1 led_driver_fifo

1 BUFG : 1 buffer d'horloge

7 CARRY4 : 7 additionneurs de 4 bits pour le compteur 28 bits

1 LUT 1 1 inverseur pour le resetn

1 inverseur pour le décodage du btn1 sur l'entrée de la fifo : fifo_in[0] = not btn1

3 LUT 2 1 LUT2 pour la logique de détection du front montant sur btn0

1 LUT2 pour le décodage du code couleur

27 LUT4: 21 LUT4 pour le compteur 28 bits

5 LUT4 pour la logique sur la machine d'état onoff

1 LUT4 pour générer le signal end_cycle

2 LUT5: 2 LUT5 pour la logique sur la machine d'état onoff

1 LUT6 2 LUT5 pour la logique sur la machine d'état onoff

31 FDCE 28 registres pour le compteur

2 registres pour la détection du front montant de btn0

1 registre pour le décalage du signal end_cycle

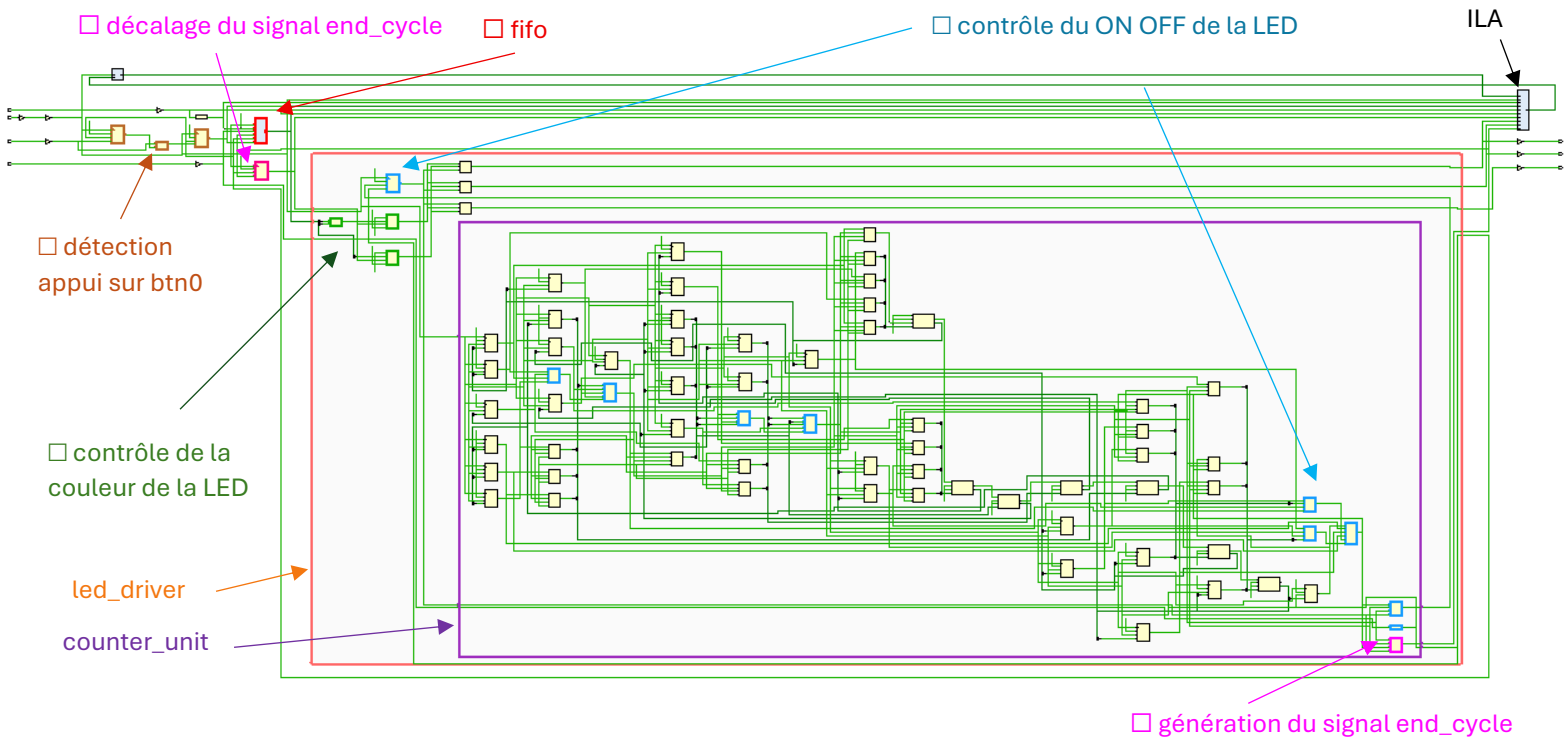
1 FDPE 1 registre pour la machine d'état onoff qui vient contrôler la couleur de la LED

2 LD 2 registre pour gérer la couleur qui vient contrôler la couleur de la LED

4 IBUF 4 buffers d'entrée pour les 4 entrées clk, resetn, btn0 et btn 1

3 OBUF 3 buffers de sortie pour les 3 sorties de contrôle de la LED : led_r, led_g, led_b

- Identification des blocs sur le schéma implémenté



- Implémentation / rapport de timings

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1,718 ns	Worst Hold Slack (WHS): 0,022 ns	Worst Pulse Width Slack (WPWS): 2,750 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 4199	Total Number of Endpoints: 4183	Total Number of Endpoints: 2370

All user specified timing constraints are met.

Le design respecte les contraintes de timings pour une horloge à 125Mhz.

Il n'y a pas de violation de setup et de hold TNS = 0 et THS= 0.

- Validation sur la carte: mesures sur l'ILA

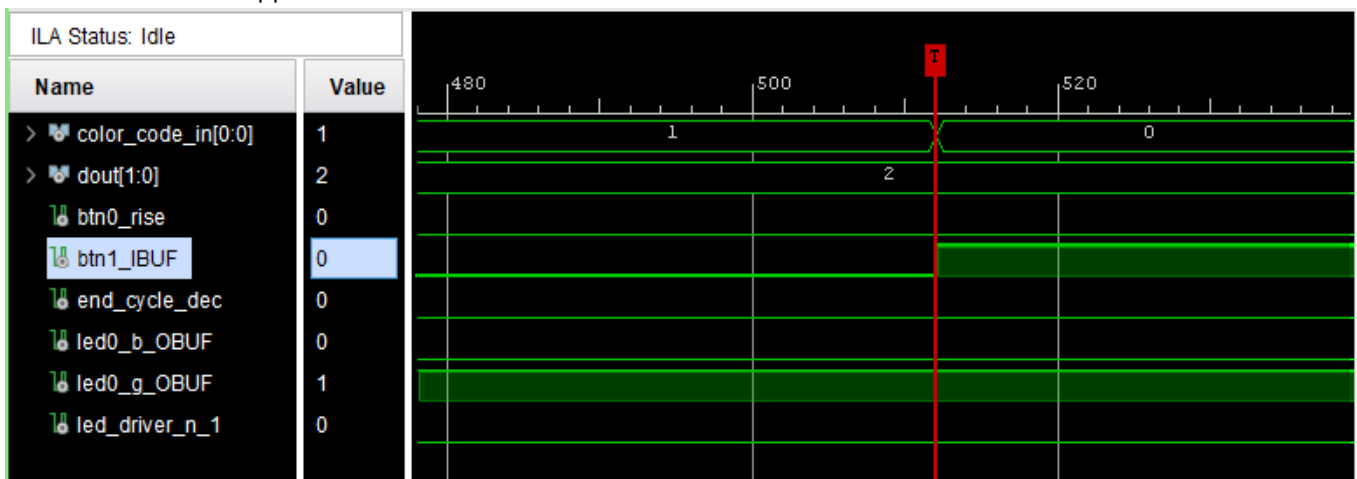
Liste des signaux :

- color_code_in[0] : bit p de poids faible en entrée de la fifo
- dout[1:0] sortie de la fifo
- btn0_rise : signal de détection du front montant sur btn0
- btn1_ibuf : signal btn1 en sortie du u buffer
- end_cycle_dec : signal end_cycle décalé d'un coup d'horloge
- led0_b_obuf, led0_g_obut : sorties de commande bleu et vert de la LED
- led_driver_n_1 : représente le signal end_cycle

Simulations faites avec une cible = 200 000 000

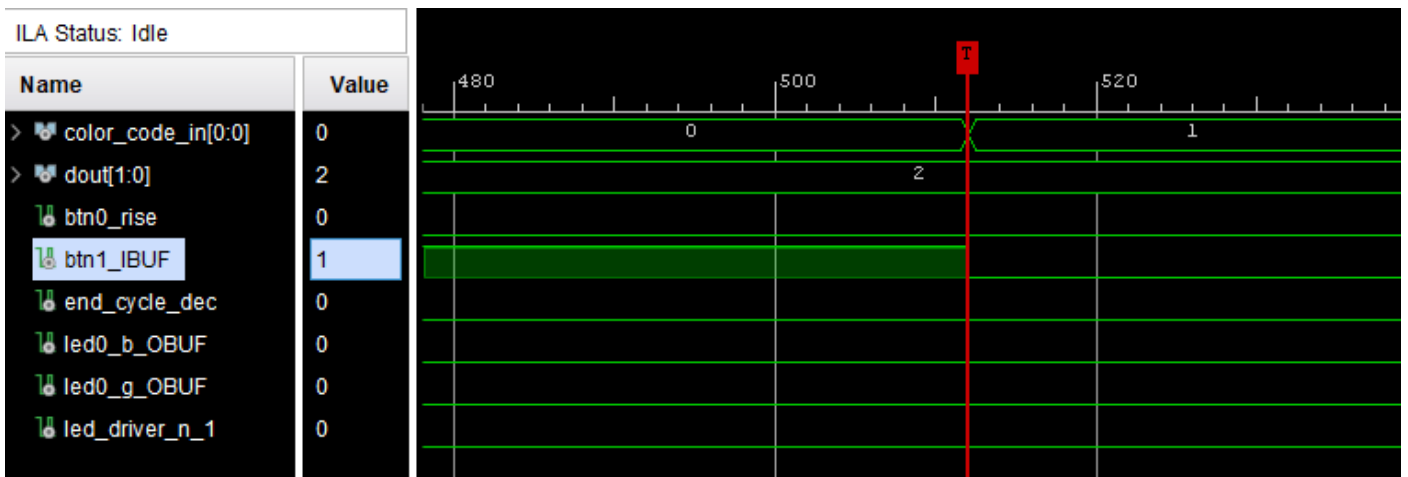
- Validation du comportement de btn1

- appui sur btn1



lors de l'appui sur btn1, l'entrée de la fifo color_code_in reçoit le nouveau code
si btn1 = 1 => color_code_in = "10" et color_code_in[0] = 0

- on relâche btn1



Lorsqu'on relâche btn1, l'entrée de la fifo color_code_in reçoit le nouveau code
si btn1 = 0 => color_code_in = "11" et color_code_in[0] = 1

○ Validation du changement de couleur

• Passage en bleu



La lecture de la fifo est effectuée quand $rd_en = end_cycle(led_driver_n_1) = 1$

La sortie de la fifo (dout) est mise à jour un coup d'horloge plus tard. La LED s'allume en bleu à ce moment-là.

• Passage en vert



La lecture de la fifo est effectuée quand $rd_en = end_cycle(led_driver_n_1) = 1$

La sortie de la fifo (dout) est mise à jour un coup d'horloge plus tard. La LED s'allume en vert à ce moment-là.

○ Validation du comportement de btn0

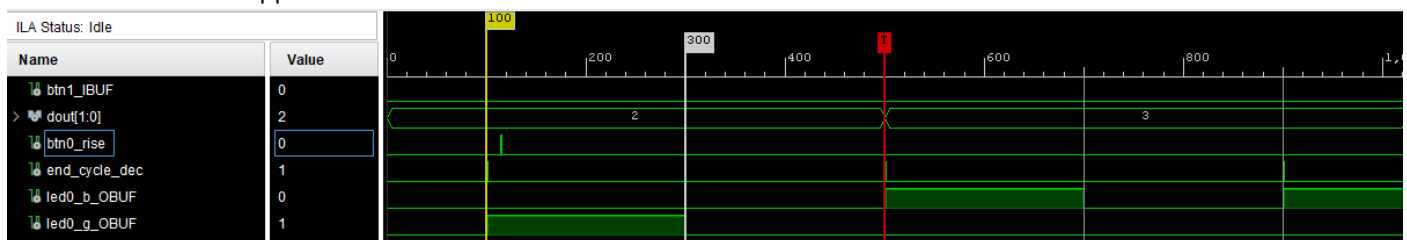
Afin de pouvoir mesurer le comportement du circuit suite à l'appui sur btn 0, la synthèse à été refaite avec une cible = 200

Avec cible = 200 et une horloge = 125Mhz le temps d'allumage de la LED est de 1.6us

L'ILA prend aussi des échantillons à une fréquence de 125MHz, il faudra 200 échantillons pour capturer un cycle d'allumage.

en échantillonnant sur une fenêtre de largeur 1024, on peut obtenir une capture du fonctionnement du circuit après un appui sur bnt0

• appui sur btn0 avec btn1 relâché



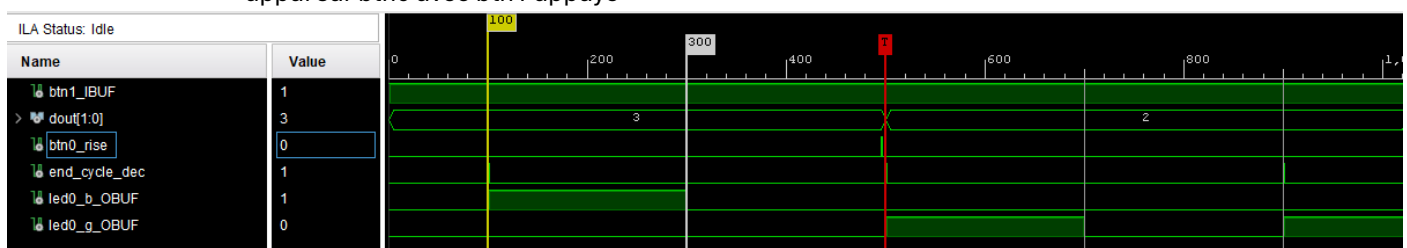
L'appui sur btn 0 déclenche une détection du front montant $btn0_rise = 1$ sur une periode d'horloge

La fifo est lue à la fin du cycle en cours : la sortie de la fifo (dout) est mise à jour

Le led_driver est mis à jour sur end_cycle_dec : la LED passe de vert à bleu.

La durée d'un cycle d'éclairage st bien de 200 échantillons.

• appui sur btn0 avec btn1 appuyé



L'appui sur btn 0 déclenche une détection du front montant $btn0_rise = 1$ sur une periode d'horloge

La fifo est lue à la fin du cycle en cours : la sortie de la fifo (dout) est mise à jour

Le led_driver est mis à jour sur end_cycle_dec : la LED passe de bleu à vert

- Validation sur la carte: mesures à l'oscilloscope

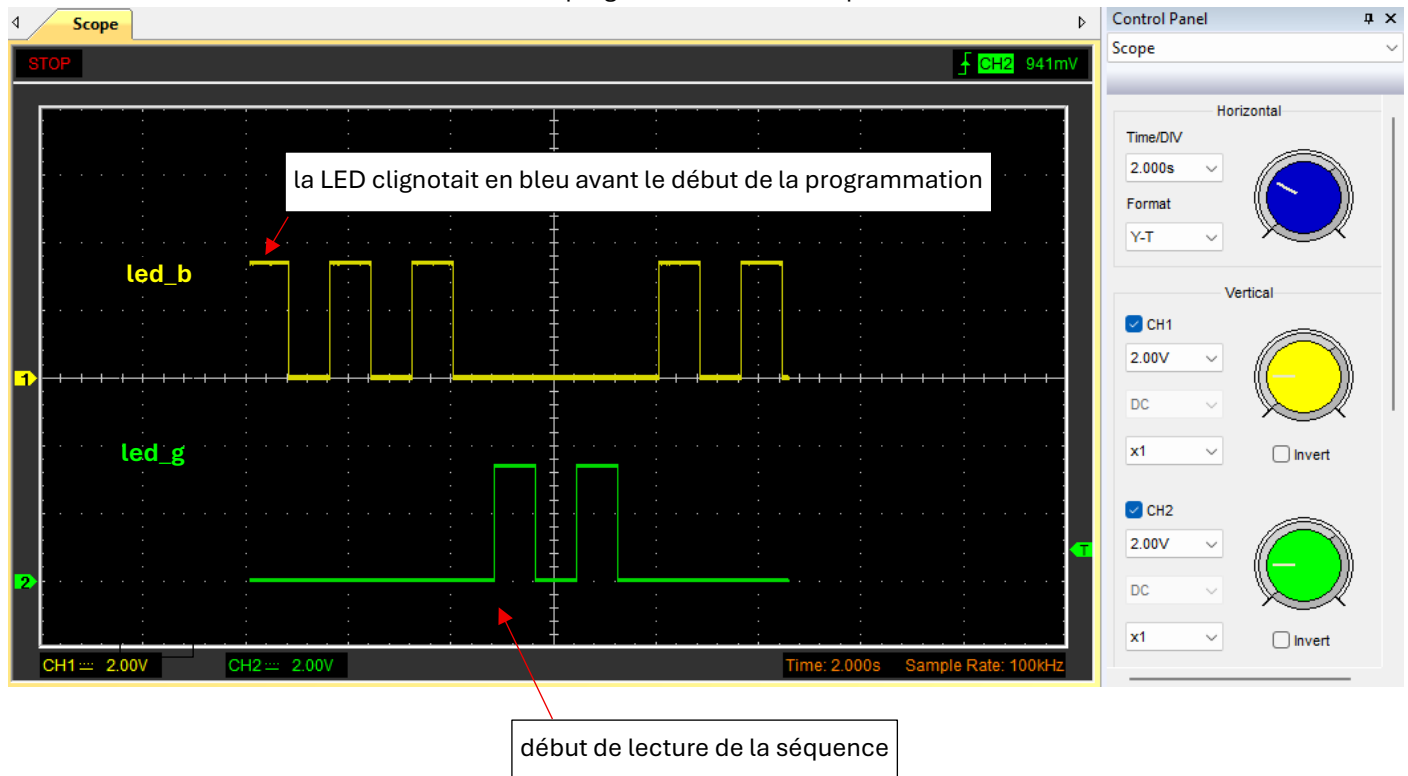
Afin de mesurer plus facilement sur l'oscilloscope:

Les sorties led_g et led_b sont dupliquées et routées sur des ports user_io.

afin de pouvoir mesurer quelques cycles sur la capacité de capture de l'oscilloscope et avoir le temps de programmer la fifo en appuyant sur les boutons , la période d'allumage est réduite à 1 seconde, le cycle de comptage est donc réduit à 100 000 000

L'implémentation est refaite avec ses paramètres.

La séquence capturée a été déclenché sur un front montant de la LED V en enregistrant une séquence dans la fifo VVBBVVBB... la LED étant bleu avant le début de programmation de la séquence



- Démonstration

Après avoir resetté le circuit, on commence la programmation de la fifo avec la séquence suivante

couleur	bnt1
vert	1
bleu	0
vert	1
vert	1
bleu	0
+bleu	0
vert	1
bleu	0
vert	1

On vérifie le déroulé de la séquence sur la LED 0

Conclusion

Le TP4 a permis de contrôler l'affichage de la LED en fonction d'une séquence programmée dans une mémoire fifo.